

Integrating Bluesky into Resonai/BossCat: Implementation Plan

1. Automated Posting

To post content from the Resonai/BossCat project to Bluesky automatically, leverage open-source Bluesky API libraries and bots. The Bluesky network uses the AT Protocol (ATProto), and several community SDKs exist for different languages. For example:

- **ATProto Libraries:** Use the official TypeScript SDK (`@atproto/api`) in a Node.js service or a Python library like `atproto` (by MarshalX) in a script. These provide high-level methods to authenticate and create posts. The Python `atproto` client supports rich text facets (for mentions/hashtags) – e.g. using a `TextBuilder` to format links and hashtags properly ¹ ². In Node.js, the ATProto SDK allows creating records (posts) via `agent.api.com.atproto.repo.createRecord` calls (after logging in with a handle and app password).
- **Bluesky CLI/Bots:** There are command-line tools and bots for Bluesky. For instance, *blue-sky-cli* (TypeScript) and *bsky.sh* (shell) clients can post from scripts ³ ⁴. The *Bluesky Earthquake Bot* (open-source) automatically posts external events ⁵, demonstrating how to integrate custom triggers. Similarly, a **GitHub Action** called **“Send Bluesky Post”** (`myConsciousness/bluesky-post`) enables CI/CD-based posting. By adding this to a GitHub Actions workflow, you can push updates to Bluesky whenever certain events occur – for example, on a new release or evidence report. The action only needs your Bluesky handle and an App Password (stored as secrets) and a text input ⁶. This makes it easy to trigger Bluesky posts from CI pipelines (e.g. via scheduled runs or when a Git commit includes a certain tag).
- **Scheduling & Templating:** For scheduled or templated posts, you can run posting scripts on a schedule (cron or GitHub Actions scheduler). The blog by Lukas Nagel describes using a simple cron job (or Kubernetes CronJob) to periodically run a Python script that fetches content (in his case, from an RSS feed) and then posts to Bluesky ⁷. In our context, this could pull from BossCat’s evidence logs or status updates. The content template can be flexible – e.g. a markdown snippet or an ECRR status that gets converted to a post. Keep posts concise due to Bluesky’s ~300-character limit, or split longer text into threads. Some third-party tools (like Fedica) even support converting long posts or Markdown into threads automatically ⁸, but you can also implement this: e.g. have the script break a Markdown report into a series of posts and post them in reply chain order to form a thread.
- **CI/CD Triggers:** Integrating with GitHub Actions or similar, you could post updates when certain project events happen. For example, after a successful deployment or a new “Report” in the ECRR workflow, an action can format a short summary and use the Bluesky Post Action to publish it. This ensures posts are tied to real project events. Using repository data also means you can include links or references (the Bluesky Action automatically handles mentions/links facets in text ⁹).

Trade-offs: The direct API approach (using ATProto libraries) gives full control and can run on your infrastructure (self-hosted). It requires storing credentials (use Bluesky App Passwords, not the main password, for security ¹⁰). GitHub Actions are convenient for repository-driven content but rely on GitHub's infrastructure. A hybrid approach could be to use GH Actions to queue up posts (in a repo or issue) and a self-hosted bot to actually publish them after approval (see review workflow below). Always ensure error handling and rate limits are respected – Bluesky's API is fast, but avoid spamming posts or you may hit service limits.

2. Following Strategy & Feed Discovery

Growing the Bluesky account's reach and ensuring it follows relevant content requires an intelligent following strategy:

- **Curate High-Signal Accounts:** Identify and follow key Bluesky users in domains like OSINT, misinformation research, fact-checking, and AI transparency. The community has already compiled "starter packs" of accounts for such topics. For example, Bluesky Starter Packs include lists for *Open Source Intelligence* – including Bellingcat contributors and other trusted OSINT researchers ¹¹. By importing these lists (many starter packs provide a one-click follow link or list of handles), the Resonai account can quickly follow a network of relevant experts. This seeding ensures your feed contains the kind of content (and hashtags) you care about.
- **Leverage Bluesky Directory/Lists:** Bluesky's ecosystem offers tools like the *Bluesky Directory* and user-created lists. These often categorize notable accounts by topic. The *Bluesky Follow Chain* tool, for instance, helps bulk-follow suggested users in a category ¹². Using such tools, you can programmatically follow a batch of users (after a one-time authorization) instead of manually searching. This can be scriptable via the API as well – e.g. writing a small script to follow accounts by DID or handle if you have a prepared list.
- **Advanced Search & Discovery Feeds:** Bluesky now supports searching posts and profiles by keywords. You can use this to discover accounts that mention certain topics. For example, searching bios or posts for "OSINT", "fact-check", "misinfo" etc., via an advanced search tool (like Fedica's web UI or the `app.bsky.feed.searchPosts` API) can surface users who talk about those topics. Searching with a `#` (hashtag) prefix finds posts containing that tag ¹³ – from there you can identify authors to follow. Additionally, subscribe to some of Bluesky's algorithmic **custom feeds** for discovery. Bluesky's default **What's Hot/Discover** feed will show trending posts network-wide. There are also user-generated feeds (via the feed generator service) that focus on specific content (for instance, feeds for tech news, or "Popular with friends"). By subscribing to these feeds in your client, the Resonai account can passively discover popular conversations and accounts, which you can then choose to follow or engage with. This approach is open and user-controlled, aligning with Bluesky's vision of decentralized algorithms ¹⁴ ¹⁵ (you can add or remove feeds anytime to tune what you see).
- **OSINT & Misinformation Communities:** Make the account known in the communities by engaging with them. Follow conversations by known OSINT tags (e.g. #OSINT, #misinformation, #factcheck). When appropriate, reply or repost content from these communities to signal that Resonai/BossCat is contributing to the discussion (not just broadcasting). Because Bluesky is smaller and community-driven, genuine engagement can lead to reciprocal follows, building a quality follower base. There

might also be Bluesky “feeds” specifically for open-source investigations (for example, if someone created a custom algorithmic feed that aggregates posts about verifications or debunks). Checking Bluesky feeds like “What’s Science” or others in the feed generator showcase ¹⁶ may yield something related (or you could create one – see next section on feed generators).

Tools & APIs: The Bluesky API can list an account’s followers or followings, so you could use that to monitor new relevant users (for instance, if a known OSINT figure joins Bluesky, a script could detect the new follower list of a key account and prompt you to follow them). There are also third-party tools like *SkyFollower Bridge* to find Twitter followers on Bluesky ¹⁷ – useful if you have a list of Twitter accounts you trust and want to see if they have Bluesky handles.

3. Hashtag & Trend Intelligence

Identifying effective hashtags on Bluesky will improve post visibility and help you tap into trending discussions. Unlike Twitter, Bluesky doesn’t have a global “trending topics” sidebar, but we can derive trends through engagement patterns and third-party trackers:

- **Popular Hashtags in Domain:** Common tags in the fact-checking/Open-Source arena include **#OSINT**, **#misinformation**, **#disinformation**, **#FactCheck**, **#AI** (for AI transparency), and project-specific tags if any (like **#ECRR** if you coin it). Because Bluesky is community-moderated, users often use hashtags to categorize content (even though the Bluesky app doesn’t hyperlink them by default – many third-party clients do, and search picks them up). As a starting point, use the search endpoint to query these tags and see how frequently they appear. For example, searching `#OSINT` will show posts that include it ¹³ – if results are frequent and recent, it’s an active tag. The Bluesky Starter Packs site descriptions also hint at tags (e.g., one pack description included “**#osint #dataviz**” for data journalists ¹⁸), indicating those are recognized tags in that community.
- **Trend Tracking via Firehose:** For real-time trend detection, consider tapping Bluesky’s **firehose** (the feed of all public posts). This requires a bit of engineering, but there are open tools to help. BlueSky provides an Event Stream API (Firehose) that developers can connect to (with appropriate access). For example, a community project built a *Bluesky Trending* tool that listens to the firehose and ranks posts by engagement in rolling 10-minute windows ¹⁹. From the top posts, it extracts the hashtags that are appearing most often ²⁰. You could implement a scaled-down version: use the Firehose (via a library like `atproto-firehose` for Node ²¹ or `Skyfall` gem in Ruby ²²) to stream posts, tally likes/reposts in a short timeframe, and flag which hashtags appear in those top posts. This would highlight emergent trending tags on Bluesky relevant to general chatter. If maintaining a live firehose connection is too heavy, an alternative is periodic polling: use `app.bsky.feed.getTimeline` or the “What’s Hot” feed API to fetch trending posts periodically, then parse out hashtags. This is less comprehensive but simpler.
- **CLI or Dashboard Tools:** If you prefer not to build from scratch, there are already tools and scripts for trend detection. The *Bluecrawler* project’s trending page ²⁰ and *Bsky Insights* show real-time hashtags – check if they provide an API or if their code is open-source. Also, the community has feed generators in progress for trends (a demo on designing a “Daily Trending Hashtags” feed generator is noted by Bluesky devs ²³). In lieu of a fully automated feed, even a CLI script that prints the top 5 hashtags in the past hour (by querying search or your collected data) could be used to guide your content (e.g., choosing which tag to include on a given day’s post for visibility).

- **Emerging Topic Alerts:** Integrating this with BossCat: if your system detects a sudden surge in discussion about a topic you cover (say, a new piece of misinformation going viral), having a hook to notify or prepare a Bluesky post is valuable. For instance, your OSINT agents might flag a trending keyword in the wild – you can map that to a Bluesky hashtag search to see if it's also trending there, and then quickly draft a post addressing it (with that hashtag). This blends with the event-hooks section below.

In summary, Bluesky's trend intelligence will rely on custom solutions (open and auditable) since the network is new. By using **open-source firehose listeners and search APIs**, you can maintain an up-to-date picture of which hashtags and topics are gaining traction. Incorporating those hashtags in your posts (when relevant) will make them more discoverable by others following or searching those trends.

4. Technical Feeds & Event Hooks

To tightly integrate Bluesky posting with the BossCat/ECRR system, set up event hooks so that notable events in your pipeline can trigger content creation. This ensures Bluesky updates are timely and context-driven:

- **ECRR Logs to Posts:** Identify which points in the ECRR (Evidence → Contain → Rollback → Report) cycle warrant a public update. Likely the **Report** stage (when you have a confirmed piece of open evidence or a conclusion) is a good trigger. You could implement a pipeline where, once a report is generated or marked ready, a script formats a brief summary (e.g., "New investigation result: [title]. Evidence suggests XYZ... #misinformation #OSINT") and either posts it automatically or places it in a queue for review. This script could run as part of your CI (for example, after a merge to a `reports` branch) or as a webhook from your internal system. Because this needs to be reviewable (see moderation below), you might actually emit a draft somewhere first.
- **BossCat OEM Agent Hooks:** If BossCat has agents monitoring certain data sources (say social media streams or internal telemetry), you can connect those to Bluesky. One approach is to use an **RSS or feed bridge**: for instance, if BossCat can output a feed of alerts (maybe via a JSON file or an RSS feed of recent findings), an external tool could read that and post updates. There's an open-source project *Bluestream* which turns Bluesky posts into an RSS feed ²⁴ – you could do the reverse by adapting its approach (have a small app that reads an RSS of BossCat alerts and posts each to Bluesky). This keeps things decoupled and reviewable (you can stop or edit the feed input before it goes out).
- **OpenTelemetry/SigNoz Integration:** For more technical signals (performance issues, errors, etc.), consider using alerting hooks. SigNoz (as an observability tool) can likely send alerts on certain conditions. Rather than blasting those to Bluesky directly (which might not be audience-friendly), you could have certain non-sensitive tech milestones post to Bluesky. Example: " BossCat system update complete – no anomalies detected in the last 24h. #AIObservability" or " BossCat detected a surge in suspicious links – investigating #OpenEvidence". These could be triggered by counters or flags in your monitoring system. Implement this by writing a small service that subscribes to your monitoring events (SigNoz or OpenTelemetry can export to webhooks). On receiving an event of interest, it prepares a post. Keep this service open-source and within your stack so you can audit it.

- **Custom Bluesky Feeds from Internal Data:** Another angle is to create a **Bluesky custom feed generator** for your project's data. Feed generators let you define an algorithmic feed (accessible via Bluesky's app as an alternate timeline). For example, you could publish a feed of "Latest Open Evidence" that composes items from your database into feed posts. Bluesky users (including your account or others) could subscribe to it. This requires hosting a small service and using Bluesky's feed generator framework (open-sourced by Bluesky). The benefit is that it doesn't spam your main account with every minor log, but still surfaces the information on Bluesky for those interested. However, implementing a custom feed is more complex than direct posting and might be overkill unless you have a lot of automated content.
- **Examples/Analog:** The concept of piping external events to social posts is already used by others. For instance, the **GitHub Trending Repos Bot** automatically posts daily trending repositories to Bluesky ²⁵. It likely runs on a schedule, pulls data from GitHub's trending API, and posts a formatted message. Likewise, the Bluesky *Earthquake Bot* listens to seismic data and posts quakes as they happen. These bots are open-source and can serve as references for structuring event-driven posting. We can mimic their patterns: have a dedicated process (could be a serverless function, a cron job, or a persistent service) listening for BossCat's triggers and then calling the Bluesky API to post.

Integration with BossCat: If BossCat is a Node.js application, it could directly import an ATProto client and fire off a post when needed (ensuring to await the response and handle errors). If it's more of a data pipeline, use message passing – e.g., BossCat agent emits an event to a queue (like Redis or a simple file drop), and a separate Bluesky bot service consumes that and posts after perhaps adding context. Keep the system **open-source and self-hosted**: all components (event listener, posting bot) should run on your infrastructure (e.g., a small VM or container) and code in your repo, so they're auditable.

Finally, **test these hooks in a non-production setting first**. You might want a private Bluesky account or a restricted feed to ensure the formatting is right and that you don't accidentally post sensitive info. Once verified, you can switch to the official account.

5. Post Review & Moderation

Maintaining a human-in-the-loop is crucial – you want to review or approve content before it goes live on Bluesky, especially given the sensitive nature of misinformation tracking. Here's how to implement a review workflow:

- **Draft Mode & Approval:** Instead of posting immediately, have your automation create a **draft post** that a human can verify. Since Bluesky doesn't (yet) have a draft API, implement this outside Bluesky. For example, your system could open a **pull request** in a repository with the proposed post content (perhaps as part of a Markdown file or in the PR body). The team can review the PR – which contains the text (and maybe a screenshot or reference if needed) – and only merge it when it's approved. The merge action can then trigger the actual post to Bluesky via the CI. This way, every automated Bluesky post goes through the normal code review process. Alternatively, use Issues or Discussions: an agent can post the draft content in a GitHub Issue comment; when someone adds a label like `ready-to-post` or comments `@cat ready-for-gate`, a webhook can catch that and trigger the post.

- **GitHub Actions Approval Gates:** GitHub Actions has a feature where jobs can require manual approval (using environments protected by reviewers). You could set up your “Post to Bluesky” step to require a certain person or team’s sign-off in the Actions UI. The workflow would pause after preparing the content, and an authorized human can then approve the job to continue, which would execute the posting step. This ensures nothing goes out without an explicit “yes”. It’s an open-source friendly approach since all logs and content live in your repo’s actions log (which is private to your org, presumably).
- **Command-Line Confirmation:** If your posting mechanism is a CLI script, simply build in a confirmation prompt. For instance, the cross-posting script used by *web3isgoinggreat.com* intentionally shows a preview of the post (including images) and waits for user confirmation before actually publishing, unless a `--no-confirm` flag is set ²⁶. You can mimic that: have your bot output the composed post (and maybe any link or image it will include) to the console or a Slack message, and require a maintainer to confirm (press “Y” or click an “approve” button) before it proceeds. This could be semi-automated: e.g., send the draft content to a private Slack channel via a bot; an admin can eyeball it and respond with a specific phrase to trigger the final publish step.
- **Integration with Evidence Workflow:** Since Resonai/BossCat might have an *evidence approval workflow* already (for example, analysts sign off on an evidence file or an internal tag like `#approved` is added), tie the Bluesky posting to that. Perhaps when an evidence item moves to a “Published” state in your system, it goes into a queue for social posting. The key is to not allow fully automated posting without a human “gate”. By aligning the social post approval with your existing approval step (like requiring an `@boss-cat approve` comment), you keep things in sync – nothing gets posted that hasn’t been vetted as accurate and appropriate to share.
- **Content Moderation and Tone:** Even after approval, maintain some guidelines for the content. Ensure the posts adhere to Bluesky’s community guidelines and the professional tone of your project. It’s wise to double-check that any external accounts you mention (via `@`) or any hashtags you use are correct and won’t lead to misinterpretation. The Bluesky Action we use automatically formats mentions and links in the post text ⁹, but you still must ensure the context is right. Human review helps catch these nuances.

In summary, **no post should go out blindly**. By using a **draft + approval** mechanism (whether through GitHub PRs, Issues commands, or a paused CI job), you keep a human reviewer in the loop. This adds a bit of friction, but it’s worth it to prevent mistakes. All the tooling in this plan (scripts, actions) can be kept in a private repository so only the team sees draft content before it’s public.

6. Cross-Posting & Visibility

Maximizing the reach of your content involves not just Bluesky but the broader decentralized social web. You can mirror posts to other platforms like Mastodon or Nostr to reach those audiences, while also boosting visibility on Bluesky itself:

- **Cross-Posting Tools:** There are open-source solutions that allow one-to-many posting. For example, Nicholas Zakas’s **Crosspost** utility (Node.js) provides a unified API to post a message to Bluesky, Mastodon, Twitter, LinkedIn, Discord, and Nostr in one go ²⁷. You configure each service’s

credentials once and then call a single `client.post("message")` to send to all. This is great for announcements or reports where you want the same content shared across your accounts. Another is the **w3igg-crossposter** (by Molly White), a Python tool that takes content (in her case a blog entry), splits it into images if too long, and posts to Twitter, Mastodon, Bluesky, Instagram, and Threads in one shot ²⁸. These tools emphasize using app-specific auth (e.g. Bluesky App Password, Mastodon token, Nostr key) and often come with rate-limit handling. For our use, we could incorporate a cross-post step so that when a Bluesky post is approved, the same content is sent to a Mastodon account (for the Fediverse) or a Nostr relay, etc. This broadens the audience for BossCat's updates without much extra effort.

- **Mastodon & Nostr Integration:** If you maintain a Mastodon account (e.g., on a Fediverse server for OSINT folks) or a Nostr identity for Resonai, mirroring Bluesky posts there can be automated. The **humanwhocodes/crosspost** library's `MastodonStrategy` and `NostrStrategy` make this straightforward ²⁹ ³⁰ – you just need your Mastodon instance URL and token, and your Nostr private key and relay URLs. Once set up, every time you call the cross-post routine, it will publish to those networks as well. Ensure you comply with each network's norms (for example, Mastodon has a content warning system for sensitive content – your crosspost tool should support adding those if needed). There's also a service called *Nootti* that cross-posts between Bluesky, Mastodon, and Nostr ³¹, but since you prefer self-hosted, using the open-source libraries or scripts is better.
- **Visibility on Bluesky:** Within Bluesky itself, some strategies to increase visibility include:
 - Using relevant **hashtags** (as noted earlier) so that people searching for those terms see your posts. This is important on Bluesky because users actively search for content via keywords due to the lack of an algorithmic global trending list.
 - Engaging with others' posts – replies and reposts can increase your account's visibility. If your Bluesky account regularly provides useful input on others' threads (especially well-known figures in misinformation/OSINT), their followers might follow you.
 - Bluesky also respects **verified domains**: if you can set up a custom handle (e.g., `resonai.com` as your handle instead of `resonai.bsky.social`), it adds credibility. People often trust and follow accounts with custom domains since it shows authenticity.
- **Custom Feeds:** Get your content into custom feeds that users subscribe to. For example, if someone makes a "Infosec News" feed generator, having your posts use certain keywords might get them picked up there. You could even collaborate to create a BossCat feed generator that others could add – but again, that's an advanced step.
- **Cross-Promotion:** Don't forget to promote the Bluesky account in your other channels. Mention in your project README, website, or newsletters that you have a Bluesky presence. Early adopters on Bluesky will likely boost your posts if they know you're there (the community is excited about new useful accounts). Cross-posting helps here – e.g., on Mastodon you might say "Follow us on Bluesky at `resonai.bsky.social` for updates" and vice-versa.
- **Monitoring & Adapting:** Finally, treat this as an iterative process. Monitor which posts get the most interaction on Bluesky (the API can fetch your post metrics). Perhaps your posts that include certain tags or media (images of evidence, etc.) do better – then do more of those. Also, keep an eye on Bluesky's feature updates: as of 2025, they may introduce official support for hashtags or a trends

feed. Stay flexible and update the automation to use new features or feeds (the project is open-source, so you can adapt quickly).

Trade-offs: Cross-posting identical content can save time, but each platform has its culture. It may be worth tailoring the tone or formatting for each. The tooling above allows conditional posting (you can choose which platforms for each message). For highly technical SigNoz alerts, maybe only Bluesky and a dedicated Mastodon bot are appropriate (Nostr might not have the audience for that, or vice versa). Be mindful not to overload any one account with too many automated posts; quality over quantity helps maintain follower trust.

Conclusion: By combining these strategies – an automated but **review-gated** posting system, smart **feed curation** and **trend monitoring**, integration of **BossCat events**, and broad **cross-platform sharing** – the Resonai Bluesky account can become a well-integrated extension of your project’s workflow. All components can be kept open-source and self-hosted: from the ATProto libraries to the CI pipelines and feed generators. This ensures transparency and control over what gets posted. With this plan, you’ll be able to efficiently publish evidence-rich updates, follow and engage with the right communities, and amplify your impact across decentralized social networks.

Sources: The implementation suggestions reference Bluesky’s official docs and community tools for API integration ¹ ⁶ , known Bluesky social analytics (trending hashtags, feeds) ²⁰ ¹⁹ , and cross-posting frameworks used in practice ²⁸ ²⁷ , among others, to ensure the solution is built on proven, open technologies.

¹ ² ⁷ ¹⁰ How to automate Bluesky posts | LNA-DEV ~ Lukas Nagel

<https://lna-dev.net/en/posts/projects/bluesky-automation/>

³ ⁴ ⁸ ¹² ¹⁶ ¹⁷ ²² ²⁵ ³¹ Community Showcase | Bluesky

<https://docs.bsky.app/showcase>

⁵ GitHub - pixelasticity/bluesky-earthquake-bot: Automatic posts to Bluesky when earthquakes happen

<https://github.com/pixelasticity/bluesky-earthquake-bot>

⁶ ⁹ Send Bluesky Post · Actions · GitHub Marketplace · GitHub

<https://github.com/marketplace/actions/send-bluesky-post>

¹¹ ¹⁸ Open Source Intelligence Bluesky Starter Packs [8+ Lists • Oct 2025]

<https://blueskystarterpack.com/open-source-intelligence>

¹³ Bluesky Advanced Search: Search Posts, Account Bios + More

<https://fedica.com/blog/bluesky-advanced-search/>

¹⁴ ¹⁵ How the Bluesky Algorithm Works While Empowering Users

<https://recurpost.com/blog/bluesky-algorithm/>

¹⁹ I made a tool to see trending posts and hashtags on Bluesky : r/BlueskySocial

https://www.reddit.com/r/BlueskySocial/comments/1izgjk/i_made_a_tool_to_see_trending_posts_and_hashtags/

²⁰ Bluesky Trending

<https://bluecrawler.com/tools/bluesky-trending>

²¹ Balloon.fyi

<https://natemay.notion.site/Balloon-fyi-141a61d06720807d844bc0dea815e5c3>

²³ Post by @marktucker.bsky.social — Bluesky

<https://bsky.app/profile/marktucker.bsky.social/post/3leczwz5mdc2x>

²⁴ GitHub - bluesky-social/atproto-ecosystem: list of projects and implementations in the AT protocol ecosystem

<https://github.com/bluesky-social/atproto-ecosystem>

²⁶ ²⁸ GitHub - molly/w3igg-crossposter: Automate crossposting web3isgoinggreat.com posts to social media

<https://github.com/molly/w3igg-crossposter>

²⁷ ²⁹ ³⁰ GitHub - humanwhocodes/crosspost: A JavaScript utility for posting across multiple social networks at once

<https://github.com/humanwhocodes/crosspost>